통신사 침해사고 기반 보안 강화 프로젝트

Oct 16, 2026 · @ChaCha

프로젝트 개요

국내 통신사의 실제 침해사고 5건을 우리 팀 프로젝트 환경(SDI)에서 인시던트 티켓 시나리오로 재현하고, 네트워크·호스트 기반 탐지와 가용성 대응을 2주 안에 구축·검증한다.

배경: 2025년 SKT 유심 정보 유출 사고는 BPFDoor 악성코드가 2021년부터 서버에 잠복했는데도 탐지되지 않았다는 점에서, 침입 차단만큼 탐지·암호화·로그가 중요하다는 것을 보여줬다.

공격 대상: 가상 통신사 고객센터 웹앱 MockTel을 직접 만들고, 사례별 취약점을 일부러 넣는다. `SECURE_MODE=0/1`로 취약 버전과 수정 버전을 전환해 같은 공격의 전후를 비교한다. 실제 통신사 이름·로고는 쓰지 않고, 프로젝트망 밖에는 배포하지 않는다.

목적

1. 실제 사고의 공격 흐름과 근본 원인을 분석한다.
2. 같은 유형의 위협을 우리 프로젝트 환경에서 안전하게 재현한다.
3. 기본 탐지로 놓치는 지점을 찾고, 탐지 규칙과 대응 절차로 보완한다.
4. 모든 과정을 수업에서 배운 인시던트 티켓 형식으로 기록한다.

핵심 질문: 기본 설정만으로는 무엇을 놓치고, 무엇을 추가해야 잡히는가?

범위

포함	제외(향후 과제)
네트워크 기반 탐지(NIDS)	물리 장비 기반 로드밸런싱
호스트 기반 탐지(HIDS)	망 분리 재설계, mTLS
가용성 침해 시나리오(소프트웨어 LB)	별도 로그 저장소 구축
인시던트 티켓 작성과 사고 대응 플레이북	MITRE ATT&CK 전체 매핑

안전 원칙

- 모든 재현과 부하 테스트는 외부와 분리된 우리 팀 프로젝트망 안에서만 한다.
- 실제 악성코드는 쓰지 않고, 무해한 기본 명령어(nc, sleep, tcpdump)로 공격 기법만 흉내 낸다.
- 실제 기업 시스템을 대상으로 하지 않으며, 사례 분석은 공개된 조사 결과만 사용한다.

성공 지표

같은 시나리오를 보강 전과 후에 각각 측정하고, 10일차에 아래 목표를 달성했는지로 프로젝트 성공을 판단한다. 보강 전 값은 2일차부터 측정해 채운다.

지표	측정 방법	보강 전	목표	관련 티켓
시나리오 탐지율	티켓별 재현 후 알림 발생 여부	측정	6건 중 6건	전체
탐지까지 걸린 시간	재현 시각부터 대시보드 알림 시각까지	측정	5분 이내	전체
오탐	평상시 1시간 동안 inc 그룹 알림 수	측정	2건 이하	전체
수정 버전에서 공격 성공	SECURE_MODE=1에서 Postman 컬렉션·재현 스크립트 실행	-	0건	INC-02, 04, 06
요청 폭주 중 오류율	같은 k6 부하	측정	1% 이하	INC-06
요청 폭주 중 응답 시간(p95)	같은 k6 부하	측정	1초 이하	INC-06
서버 1대 장애 시 오류율	k6 부하 중 web-01 정지	측정	1% 이하	INC-06
티켓 완성도	템플릿 13개 항목을 모두 채웠는가	-	6건 모두	전체

목표 수치는 첫 측정 뒤 3일차 점검에서 팀이 다시 정할 수 있다. 바꾼 경우 이 표에 사유를 남긴다.

분석 대상 사례

다섯 사례는 침투 경로가 서버, 공급망, 웹/API, 장비, 운영 절차로 겹치지 않아, 보안 계층을 고르게 점검하는 틀이 된다.

사례	사고 개요	근본 원인	교훈
SKT 유심 정보 유출 (2025)	BPFDoor가 핵심 서버(HSS)까지 확산, 유심 인증키 유출	수년간 탐지 실패, 평문 계정 정보, 키 미암호화, 로그 부족	탐지·암호화·로그가 함께 있어야 한다

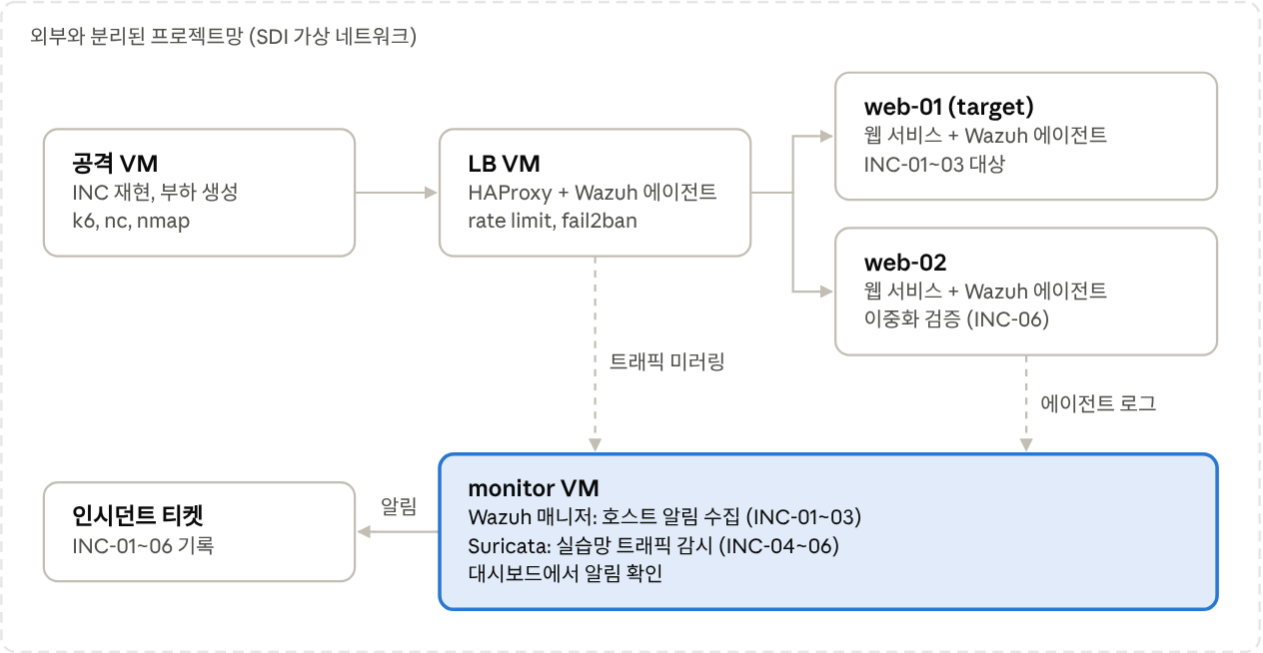
사례	사고 개요	근본 원인	교훈
SK커뮤니케이션즈 해킹 (2011)	알집 업데이트 서버 경유로 사내 PC 감염, 약 3,500만 명 유출	공급망 공격, 내부 PC에서 DB 직접 접근	의존성 무결성과 망 분리
KT 고객정보 유출 (2012·2014)	조회 시스템·홈페이지 대량 조회	대량 요청 미탐지, 객체 단위 권한 검사 부재	Rate limit과 인가 검사
KT 불법 펨토셀 소액결제 (2025)	관리되지 않는 장비가 망에 접속	기기 인증과 폐기 장비 관리 부재	망에 붙는 기기를 인증하고 감시
LG유플러스 유출·디도스 (2023), 계정정보 유출 의혹 (2025)	고객 정보 유출과 서비스 장애, 대응 중 서버 정리	레거시 관리 미흡, 가용성 대비 부족, 증거 보존 미흡	이중화와 증거 우선 보존

수치와 경위는 문서 끝 출처의 공개 보도를 기준으로 했다. SKT는 서버 28대 감염·유심 정보 25종(IMSI 기준 약 2,696만 건)·과징금 약 1,348억 원, SK커뮤니케이션즈는 약 3,495만 건, KT는 2012년 870만·2014년 1,200만 명, LG유플러스 2023은 약 29만 7천 건이다.

프로젝트 구조

프로젝트 환경은 외부와 분리된 가상 네트워크 안의 VM 5대로 구성하고, 호스트·네트워크·가용성 세 축의 탐지를 monitor VM 한 곳에 모은다. 물리 로드밸런서는 없으므로 LB VM의 HAProxy가 그 역할을 대신한다.

모든 탐지는 monitor VM으로 모여 티켓이 된다



프로젝트 환경 구성 · VM 5대

공격 VM의 요청은 LB를 거쳐 웹 서버 2대로 나뉘고, 호스트 로그와 미러링된 트래픽은 monitor VM에서 알림이 된 뒤 티켓으로 기록된다. VM이 부족하면 web-02와 LB는 Docker 컨테이너로 대신한다.

탐지 축	도구	담당 티켓
호스트 기반(HIDS)	Wazuh 에이전트·매니저, FIM, 명령 모니터링	INC-01, 02, 03
네트워크 기반(NIDS)	Suricata, arpwatch	INC-04, 05
가용성	HAProxy/Nginx, fail2ban, k6	INC-06

인시던트 티켓 시나리오

사례마다 티켓 하나를 배정하고, INC-01부터 순서대로 진행한다. 호스트 3건, 네트워크 2건, 가용성 1건이다.

티켓	근거 사례	탐지 축	재현 상황	탐지 도구	ATT&CK
INC-01	SKT 2025	호스트	숨겨진 프로세스와 패킷 감청 소켓(BPFDoor 흉내)	Wazuh 명령 모니터링	T1036, T1070.004, T1205.002
INC-02	SKT 2025	호스트	MockTel의 평문 비밀번호 저장과 하드코딩된 키	Wazuh 규칙, gitleaks	T1552.001

티켓	근거 사례	탐지 축	재현 상황	탐지 도구	ATT&CK
INC-03	SK커뮤니케이션즈 2011	호스트	시스템 바이너리·설정 파일 변조(악성 업데이트 흉내), MockTel의 취약한 구버전 의존성	Wazuh FIM, pip-audit	T1195.002
INC-04	KT 2012·2014, LG유플러스 2023	네트워크	MockTel 고객 조회 API 대량 조회(IDOR), 기본 관리자 계정, 스캐닝	Suricata 규칙	T1190, T1595
INC-05	KT 2025	네트워크	허가되지 않은 VM이 내부망에 접속	arpwatch, Suricata	T1200
INC-06	LG유플러스 2023	가용성	MockTel로 HTTP 요청 폭주	Suricata threshold, 웹 서버 로그	T1498, T1499

보안 기준 매핑 (ISMS-P)

티켓마다 우리가 한 조치를 국내 인증기준인 ISMS-P 항목에 연결해, 실무 기준에 맞춘 조치임을 보여준다.

티켓	우리 조치	ISMS-P 인증기준
INC-01	숨은 프로세스·패킷 소켓 감시, 이상 프로세스 알림	2.10.9 악성코드 통제, 2.11.3 이상행위 분석 및 모니터링
INC-02	비밀번호 해싱, 비밀값을 코드에서 분리	2.5.4 비밀번호 관리, 2.7.1 암호정책 적용, 2.7.2 암호키 관리
INC-03	파일 무결성 감시, 의존성 취약점 점검	2.9.1 변경관리, 2.10.8 패치관리, 2.11.2 취약점 점검 및 조치
INC-04	객체 단위 권한 검사, 기본 계정 제거, rate limit	2.5.1 사용자 계정 관리, 2.5.5 특수 계정 및 권한 관리, 2.6.3 응용프로그램 접근
INC-05	미인가 장비 탐지, 서버 접근을 등록된 IP로 제한	2.6.1 네트워크 접근, 2.6.2 정보시스템 접근
INC-06	이중화, 요청 제한, 장애 대응	2.9.2 성능 및 장애관리, 2.12.1 재해, 재난 대비 안전조치
공통	감사 로그, 시간대 통일, 로그 검토	2.9.4 로그 및 접속기록 관리, 2.9.5 로그 및 접속기록 점검, 2.9.6 시간 동기화
공통	플레이북, 테이블탑 훈련, 증거 보존	2.11.1 사고 예방 및 대응체계 구축, 2.11.4 사고 대응 훈련 및 개선, 2.11.5 사고 대응 및 복구

항목 번호와 이름은 IT위키의 ISMS-P 세부 점검 항목 정리본을 따랐다. 최종 보고서에 쓰기 전에 KISA가 발행한 인증기준 원문과 대조한다.

사전 준비물 체크

1일차 전에 아래를 모두 확인한다. 하나라도 안 되면 오른쪽 대안을 미리 정해 둔다. VM은 호스트 전용 네트워크로 연결되므로 PC 한 대에 모두 띄우는 것을 원칙으로 한다. PC 사이에는 이 네트워크가 이어지지 않는다.

항목	기준	안 될 때 대안
VM을 띄울 PC의 RAM	여유 16GB 이상(VM 합계 약 15GB), 권장 32GB	web-02와 lb-01을 Docker 컨테이너로 대체
CPU	8코어 이상, BIOS에서 가상화(VT-x/AMD-V) 켜짐	BIOS 설정 변경
디스크 여유	150GB 이상	외장 SSD
하이퍼바이저	VirtualBox 7.x	VMware Workstation (호스트 전용을 VMnet으로)
OS 이미지	Ubuntu 24.04 Server ISO, 팀 공용 폴더에 1개	–
Git 저장소	비공개 저장소, 전원 초대, 하네스 파일 커밋	–
팀 비밀번호 관리자	Wazuh admin 비밀번호 보관용	–
팀원 PC 도구	Postman CLI, gitleaks, pip-audit, Git	Postman 앱
화면 녹화	OBS 등 녹화 도구(발표 백업용)	Windows 기본 녹화

Windows에서 Hyper-V나 WSL2가 켜져 있으면 VirtualBox가 느려질 수 있다. 0일차에 VM 하나를 미리 띄워 속도를 확인한다.

2주 로드맵

평일 10일 동안 INC-01을 3일차까지 끝내고, 같은 환경 위에서 나머지 티켓 5건을 이어서 진행한다. 점검 지점은 3일차(INC-01 완료), 5일차(호스트 티켓 완료), 10일차(최종 보고) 세 번이다.

일차	할 일	산출물	완료 기준	상태
1	VM 5대 생성·공통 설정, 모든 패키지 설치(가이드 1~7단계), Wazuh 에이전트 등록, 역할 확정	구성도, 자산 목록	에이전트 3대 Active, Suricata·HAProxy 동작	시작 전
2	MockTel 뼈대·기능 파일 틀 배포와 패키지(wheels) 확보 → 인터넷 차단·baseline 스냅샷, INC-01 A·B·C를 기본 탐지만으로 재현	MockTel 화면, baseline 스냅샷, 탐지 공백 캡처	A만 탐지되고 B·C는 놓침을 확인	시작 전
3	INC-01 커스텀 설정·규칙 추가, 재현, 티켓 작성	INC-01 티켓	3/3 탐지	시작 전
4	INC-02 MockTel 평문 비밀번호·하드코딩 키, INC-03 파일 변조·구버전 의존성 재현	INC-02·03 티켓	두 시나리오 모두 알림 발생	시작 전
5	Suricata 커스텀 규칙 작성·검증(설치는 1일차에 완료), 1주차 점검	규칙 파일	테스트 트래픽이 탐지됨	시작 전
6	INC-04 MockTel 고객 조회 API 대량 조회·기본 관리자 계정, INC-05 미인가 VM 재현	INC-04·05 티켓	두 시나리오 모두 알림 발생	시작 전
7	INC-06 기준선 측정, 요청 폭주 재현	응답 시간·오류율 기록	서비스 저하 지점 확인	시작 전
8	rate limit, fail2ban, HAProxy 이중화 적용 후 재측정	INC-06 티켓, 전후 비교	서버 1대를 꺼도 서비스 유지	시작 전
9	티켓 6건 검토, 플레이북 작성, 30분 테이블탑 훈련	플레이북, 훈련 기록	훈련이 플레이북대로 끝남	시작 전
10	최종 보고서와 발표 자료, 남은 위험 정리	보고서, 발표 자료	발표 리허설 완료	시작 전

일정이 밀리면 9일차 테이블탑 훈련과 INC-05를 먼저 줄이고, INC-01과 INC-06은 끝까지 지킨다.

역할 분담과 협업 규칙

4명 기준으로 탐지 축마다 한 명씩 맡고, 1명은 티켓과 문서를 책임진다. 담당자 칸은 팀에서 채운다.

역할	담당 티켓	주요 작업	담당자
호스트 탐지	INC-01, 02, 03	Wazuh 설치·규칙, 호스트 시나리오 재현	
네트워크 탐지	INC-04, 05	Suricata 설치·규칙, arpwatch	
가용성·LB	INC-06	부하 측정, rate limit, HAProxy 이중화	
티켓·문서	전체	티켓 관리, 플레이북, 보고서·발표	

협업 규칙

- 티켓은 사고를 재현한 직후 바로 작성한다. 몰아서 쓰면 타임라인이 무너진다.
- 모든 시나리오는 보강 전과 후를 모두 캡처해 공유 폴더에 남긴다.
- 하루를 마칠 때 로드맵의 상태 칸을 갱신한다.
- 설정 파일과 규칙 파일은 Git으로 관리하고, 비밀값은 저장소에 올리지 않는다.

INC-01 상세 진행 계획

INC-01은 "포트 기반 점검만으로는 BPFDoor류 백도어를 놓친다"는 것을 재현하고, 보강 후 3/3 탐지를 보여주는 것이 목표다.

단계	흉내 낼 BPFDoor 기법	재현 명령(무해)	기본 탐지	보강 후
A	포트를 여는 단순 백도어	`nc -lvnp 4444 &`	탐지(규칙 533)	탐지
B	/dev/shm에서 실행 후 파일 삭제, 이름 위장	sleep을 /dev/shm에 복사·실행 후 삭제	놓침	탐지(규칙 100110)
C	포트 없이 BPF 필터로 패킷 감청	`tcpdump -i any 'udp port 9999' -w /dev/null &`	놓침	탐지(규칙 100111)

진행 순서

1. target VM에 Wazuh 에이전트, monitor VM에 Wazuh 매니저를 설치한다.
2. 기본 설정 상태에서 A·B·C를 재현하고, B·C가 잡히지 않는 화면을 캡처한다.
3. 에이전트에 명령 모니터링 3개를, 매니저에 규칙 2개를 추가하고 재시작한다.
4. A·B·C를 다시 재현하고, 대시보드에서 알림 3건을 확인한다.

5. 프로세스를 종료하기 전에 증거를 보존한다(`/proc/<PID>/exe` 복사, ss·ps 출력 저장).
6. 재현 프로세스를 정리하고 INC-01 티켓을 작성한다.

에이전트 설정 (target VM, `/var/ossec/etc/ossec.conf`, 주기 60초)

```
<localfile>
  <log_format>full_command</log_format>
  <command>ls -l /proc/[0-9]*/exe 2>/dev/null | grep -E '\
(deleted\)|/dev/shm/|/tmp/'</command>
  <alias>inc01_suspicious_exe</alias>
  <frequency>60</frequency>
</localfile>

<localfile>
  <log_format>full_command</log_format>
  <command>ss -0 -p | sort</command>
  <alias>inc01_packet_sockets</alias>
  <frequency>60</frequency>
</localfile>
```

기본 설정에 있는 `netstat listening ports` 항목의 주기도 60초로 줄이고, `net-tools` 패키지를 설치한다.

매니저 규칙 (monitor VM, `/var/ossec/etc/rules/local_rules.xml`)

```
<group name="inc01,">
  <rule id="100110" level="12">
    <if_sid>530</if_sid>
    <match>ossec: output: 'inc01_suspicious_exe'</match>
    <regex type="pcre2">\(deleted\)|/dev/shm/|/tmp/</regex>
    <description>INC-01: 삭제되었거나 임시 경로에서 실행 중인 프로세스</description>
    <mitre><id>T1070.004</id><id>T1036</id></mitre>
  </rule>
  <rule id="100111" level="10">
    <if_sid>530</if_sid>
    <match>ossec: output: 'inc01_packet_sockets'</match>
    <check_diff />
    <description>INC-01: 패킷 소켓 목록 변경 (BPF 백도어 의심)</description>
    <mitre><id>T1205.002</id></mitre>
  </rule>
</group>
```

B 단계 재현 명령

```
sudo cp /usr/bin/sleep /dev/shm/dbus-helper && sudo /dev/shm/dbus-helper 3600
& sleep 1; sudo rm /dev/shm/dbus-helper
```

정리 명령

```
sudo pkill -f dbus-helper; sudo pkill tcpdump; pkill nc
```

티켓 템플릿과 최종 산출물

모든 티켓은 아래 항목을 채우며, 특히 "증거 보존"은 LG유플러스 2025 사례의 교훈에 따라 빠짐없이 쓴다. 수업에서 쓴 양식이 있으면 그 양식을 우선한다.

```
티켓 ID     : INC-0X
제목        :
근거 사례   :
탐지 시각   :
탐지 경로   : (NIDS / HIDS / 서비스 로그, 규칙 ID)
심각도      : (치명 / 높음 / 중간 / 낮음)
영향 자산   :
ATT&CK      :
타임라인    : 탐지 → 분류 → 격리 → 증거 보존 → 제거 → 복구
증거 보존   : (무엇을, 어디에 저장했는가)
조치 내용   :
결과        : (보강 전후 비교)
상태        : (열림 / 대응 중 / 종결)
교훈        :
```

최종 산출물

- [ ] 프로젝트 환경 구성도와 자산 목록
- [ ] 인시던트 티켓 6건(INC-01~06)
- [ ] Wazuh·Suricata 설정과 규칙 파일
- [ ] 가용성 전후 비교 자료(응답 시간, 오류율)
- [ ] 사고 대응 플레이북과 테이블탑 훈련 기록
- [ ] 최종 보고서와 발표 자료

리스크와 대응

리스크	대응
Wazuh 설치가 무겁거나 실패함	1일차에 바로 설치, 막히면 OSSEC이나 auditd로 교체
VM 리소스 부족	웹 서버 이중화는 Docker 컨테이너 2개로 대체
부하 테스트가 외부에 영향을 줌	외부와 분리된 내부 네트워크에서만 실행
일정 지연	INC-05와 테이블탑 훈련부터 축소

리스크	대응
사례 수치가 부정확함	보고서 작성 전에 공식 발표 원문으로 확인

발표 데모 시나리오

10분 시연에서는 핵심 장면 3개만 라이브로 보여주고, 나머지는 성공 지표 표와 티켓으로 대신한다. 리허설은 9일차에 한다.

1. **환경 소개 (1분)**: 네트워크 구성도로 VM 구성과 탐지 흐름을 설명한다.
2. **INC-01 (3분)**: 기본 탐지만 켜고 B·C를 재현해 알림이 없는 화면을 보여준 뒤, 규칙을 적용해 3/3 탐지되는 대시보드를 보여준다.
3. **INC-04 (2분)**: 같은 Postman 컬렉션을 SECURE_MODE=0과 1에서 실행해, 남의 고객 정보 조회가 200에서 403으로 바뀌는 것을 보여준다.
4. **INC-06 (3분)**: k6 부하를 걸어 둔 채 web-01을 끄고, 서비스가 유지되는 그래프를 보여준다.
5. **결과 (1분)**: 성공 지표 표의 보강 전후 값을 보여준다.

백업: 9일차까지 2~4번 장면을 모두 녹화해 둔다. 라이브가 막히면 바로 녹화본으로 넘어간다. 발표장 PC에서 VM을 띄울 수 있는지도 미리 확인한다.

한계와 향후 과제

이 프로젝트는 2주 동안 작은 환경에서 우리가 아는 시나리오를 검증한다. 발표에서 아래 한계를 먼저 밝힌다.

한계	영향	향후 과제
BPFDoor의 일부 은신 기법만 흉내 낸다(매직 패킷, 역방향 연결 없음)	실제 변종에 대한 탐지력은 검증하지 못한다	KISA 등 공개 점검 가이드의 지표로 규칙 추가
시나리오를 아는 팀이 탐지 규칙을 만든다	규칙이 시나리오에 맞춰질 수 있다	다른 팀이 모르는 시나리오로 공격하는 레드팀 평가
단일 /24 대역, VM 6대	망 분리와 횡적 이동 차단 효과를 측정하기 어렵다	관리망·서비스망·DB망 분리
부하 규모가 실제 디도스보다 훨씬 작다	가용성 결론을 일반화하기 어렵다	회선 단위 방어(클라우드 디도스 대응 서비스) 검토
로그가 monitor 한 곳에만 모인다	monitor가 뚫리면 로그도 지워질 수 있다	수정할 수 없는 원격 로그 저장소
사례 정보는 공개 보도에 기대고 있다	실제 내부 원인과 다를 수 있다	정부 조사 보고서 원문으로 보완

한계	영향	향후 과제
기간이 2주다	INC-05와 테이블탑 훈련이 줄어들 수 있다	후속 학기에 확장

출처

아래 자료는 Oct 16, 2026 기준으로 확인했다. 사례 수치는 대부분 언론 보도 기준이므로, 최종 보고서 전에 과학기술정보통신부·개인정보보호위원회 보도자료 원문과 국가법령정보센터 조문으로 다시 확인한다.

구분	내용	출처
SKT 2025	민관합동조사단 최종 결과(2025.7.4): 서버 28대 감염, 악성코드 33종, 유심 정보 25종·IMSI 기준 약 2,696만 건 유출	디지털데일리
SKT 2025	개인정보위 과징금 1,347억 9,100만 원(2025.8.27), 유심 인증키 평문 저장 지적	바이라인네트워크, 머니투데이
SK커뮤니케이션즈 2011	업데이트 파일 변조로 사내 PC 62대 감염, 약 3,495만 건 유출	경향신문, 시사경제
KT 2012·2014	2012년 조회 프로그램으로 870만 명 유출, 2014년 홈페이지 해킹 1,200만 명 유출·하루 최대 31만 건 접속 미탐지	이투데이, 메트로서울, 비즈워치
KT 2025	불법 펨토셀 무단 소액결제, 중간 조사 결과(2025.11.6), 정부 결론(2025.12)	뉴시스, 바이라인네트워크, 헤럴드경제
LG유플러스 2023	29만 7,117건 유출, 과징금 68억 원(2023.7.12), 단종 소프트웨어를 쓴 고객인증시스템	바이라인네트워크
LG유플러스 2025	서버 접근제어 솔루션 해킹·계정정보 유출, 제보 뒤 서버 폐기·OS 재설치로 수사 의뢰	헤럴드경제, 바이라인네트워크
법령	정보통신망법 제48조의3 침해사고 신고(24시간, 2026.10.1 시행)	IT위키, 국가법령정보센터
법령	개인정보 유출 신고 72시간	보안뉴스
기준	ISMS-P 인증기준 세부 점검 항목	IT위키
기준	NIST SP 800-61 Rev.3 (2025.4)	NIST CSRC
도구	Wazuh 4.14 설치·사양, Suricata 설치	Wazuh 문서, Suricata 문서

티켓 상세 (INC-02~06)

INC-02~06을 INC-01과 같은 형식으로 정리했다. 모든 규칙 ID와 SID는 하네스 `03-id-registry.md`의 티켓 대역을 따르고, 재현은 프로젝트망 안에서 무해한 기본 명령으로만 한다.

티켓	담당	진행일	MockTel 대상	탐지 번호
INC-02	호스트 탐지	4일차	`POST /login`, `GET /debug/config`	Wazuh 100120–100122
INC-03	호스트 탐지	4일차	앱 파일, `requirements-vuln.txt`	Wazuh 100130–100131
INC-04	네트워크 탐지	6일차	`GET /api/customers/<id>`, `POST /admin/login`	SID 1000401–1000402, Wazuh 100140
INC-05	네트워크 탐지	6일차	(앱 아님) rogue VM	SID 1000501, Wazuh 100150
INC-06	가용성·LB	7~8일차	`GET /search?q=`	SID 1000601, Wazuh 100160–100161

INC-02: 평문 비밀번호와 하드코딩된 키

근거: SKT 2025. 서버 파일에 계정 정보가 평문으로 있었고, 유심 인증키가 암호화되지 않은 채 저장돼 있었다.

구분	SECURE_MODE=0 (취약)	SECURE_MODE=1 (수정)
`auth_users` 테이블	비밀번호 평문 저장	argon2 해시 저장
`config_leak.py`	DB 비밀번호·API 키가 코드에 있음(`LAB-FAKE-` 값)	환경 변수에서 읽음
`GET /debug/config`	설정값을 그대로 보여줌	404

재현

```
# attacker: 노출된 설정 페이지 조회
curl -s http://lb-01/debug/config
# web-01: DB에 평문 비밀번호가 있는지 확인
sqlite3 /opt/mocktel/data/mocktel.db "select count(*) from auth_users where
password not like '\$argon2%';"
```

탐지

번호	방식	내용
100120	명령 모니터링 (alias `inc02_hardcoded_secrets`)	앱 코드에서 `LAB-FAKE-` 나 `password =` 패턴 발견
100121	명령 모니터링 (alias `inc02_plaintext_pw`)	해시가 아닌 비밀번호 개수가 0보다 큼
100122	Nginx 접근 로그	`/debug/config` 요청 발생

```
<!-- web-01 에이전트 -->
<localfile>
  <log_format>full_command</log_format>
  <command>grep -rEn "LAB-FAKE-|password *= *['\"]" /opt/mocktel/app --
include=*.py</command>
  <alias>inc02_hardcoded_secrets</alias>
  <frequency>300</frequency>
</localfile>
```

팀원 PC에서는 저장소에 `gitleaks detect --source .` 를 돌려 같은 비밀값이 Git 히스토리에 남았는지 확인한다.

기대 결과: SECURE_MODE=0에서 3개 규칙이 모두 알림을 내고, 1에서는 `/debug/config` 가 404가 되며 평문 비밀번호 개수가 0이 된다.

INC-03: 파일 변조와 구버전 의존성

근거: SK커뮤니케이션즈 2011. 공개 소프트웨어의 업데이트 파일이 악성 파일로 바뀌어 사내 PC 62대가 감염됐다.

재현 (악성 업데이트 흉내, 내용은 무해한 주석)

```
# web-01
echo "# INC-03 tamper test" | sudo tee -a /opt/mocktel/app/features/search.py
sudo cp /usr/bin/true /usr/local/bin/mocktel-updater
```

탐지

번호	방식	내용
기본 550·554	Wazuh FIM	파일 변경, 새 파일 추가
100130	550을 능가하는 팀 규칙	`/opt/mocktel` 안 파일이 바뀌면 레벨 12로 올림
100131	554를 능가하는 팀 규칙	`/usr/local/bin`에 새 실행 파일이 생김

```
<!-- web-01 에이전트: inc03_syscheck.xml -->
<syscheck>
  <directories realtime="yes"
check_all="yes">/opt/mocktel/app,/etc/nginx,/usr/local/bin</directories>
</syscheck>
```

의존성: `requirements-vuln.txt`에 취약점이 알려진 구버전 패키지 1개를 고정한다(예: `Jinja2==2.10`). 팀원 PC에서 `pip-audit -r app/requirements-vuln.txt`로 취약점이 보고되는지, `requirements.txt`로는 0건인지 확인한다. 패키지 선택과 Flask 호환성은 가용성·LB 담당이 2일차 전에 정한다.

기대 결과: 변조 후 1분 안에 100130·100131 알림이 뜨고, pip-audit은 취약 버전에서만 결과를 낸다.

INC-04: 고객 조회 API 대량 조회와 기본 관리자 계정

근거: KT 2014. 특정 IP에서 하루 최대 31만 건 넘게 접속했는데도 보안장비가 탐지하지 못했다. LG유플러스 2023은 단종된 소프트웨어를 쓰는 고객인증시스템에서 유출됐다.

구분	SECURE_MODE=0	SECURE_MODE=1
`GET /api/customers/<id>`	로그인만 하면 다른 고객 ID도 200	본인이 아니면 403, 초당 제한 초과 시 429
`POST /admin/login`	기본 계정 admin/admin 동작	기본 계정 제거, 5회 실패 시 잠금

재현

```
# attacker: 일반 고객 계정으로 로그인한 뒤 ID를 바꿔 가며 조회
for i in $(seq 1 200); do curl -s -b cookie.txt -o /dev/null -w "%
{http_code}\n" http://lb-01/api/customers/$i; done | sort | uniq -c
```

탐지 (Suricata `inc04.rules`, 탐지 결과는 Wazuh로 모인다)

```
alert http any any -> $HOME_NET 80 (msg:"INC-04: customer API bulk access";
flow:to_server,established; http.uri; content:"/api/customers/"; startswith;
threshold:type both, track by_src, count 30, seconds 10; sid:1000401; rev:1;)
alert http any any -> $HOME_NET 80 (msg:"INC-04: admin login burst";
flow:to_server,established; http.method; content:"POST"; http.uri;
content:"/admin/login"; threshold:type both, track by_src, count 5, seconds
60; sid:1000402; rev:1;)
```

Wazuh 100140은 Suricata 알림(기본 규칙 86601) 중 서명이 `INC-04`로 시작하는 것의 레벨을 12로 올린다.

Postman 자동 테스트: `tests/postman/`에 MockTel 보안 테스트 컬렉션을 둔다. 고객 A로 로그인 → 고객 B 조회, admin/admin 로그인, 연속 요청 3가지를 검사하고, 환경 파일 2개(secure0, secure1)로 같은 컬렉션을 두 번 돌린다.

```
postman collection run tests/postman/mocktel-security.postman_collection.json
-e tests/postman/secure1.postman_environment.json
```

기대 결과: SECURE_MODE=0에서 200이 대부분이고 1000401 알림이 뜨며, 1에서는 본인 외 조회가 모두 403·429가 되고 Postman 테스트가 전부 통과한다.

INC-05: 미인가 VM의 프로젝트망 접속

근거: KT 2025. 불법 펨토셀이 망에 붙어 인증정보를 빼냈고, 조사단은 인증서 관리와 비정상 접속 관리 미흡을 지적했다.

재현

1. attacker를 복제해 rogue를 만들고, VirtualBox에서 MAC 주소를 새로 발급한다.
2. IP를 `10.10.10.99`로 설정하고 lab-net에 연결한다.
3. `ping -c 3 web-01`, `curl http://lb-01/health`를 보낸다.

탐지

번호	방식	내용
100150	arpwatch 로그	`new station` (처음 보는 MAC·IP 쌍)
1000501	Suricata	등록되지 않은 주소에서 오는 트래픽

```
# monitor /etc/suricata/suricata.yaml → vars.address-groups
LAB_KNOWN: "
[10.10.10.1,10.10.10.10,10.10.10.20,10.10.10.31,10.10.10.32,10.10.10.50]"
```

```
alert ip !$LAB_KNOWN any -> $HOME_NET any (msg:"INC-05: traffic from
unregistered address"; threshold:type limit, track by_src, count 1, seconds
300; sid:1000501; rev:1;)
```

보강: 서버 ufw에서 SSH·에이전트 포트를 등록된 IP만 허용하도록 좁힌다.

기대 결과: rogue가 켜지고 1분 안에 100150과 1000501 알림이 뜨고, 보강 후에는 rogue에서 web-01로 가는 SSH가 막힌다.

INC-06: MockTel 요청 폭주

근거: LG유플러스 2023. 유출과 함께 디도스로 인한 서비스 장애가 있었다.

구분	SECURE_MODE=0	SECURE_MODE=1
`GET /search?q=`	전체 고객 테이블을 매번 훑음	페이지당 20건, 같은 검색어는 60초 캐시
lb-01	제한 없음	IP당 10초에 50회 초과 시 429, fail2ban 차단

재현 (NAT를 끊은 상태에서만)

```
// scenarios/INC-06/load.js (attacker에서 k6 run load.js)
import http from 'k6/http';
export const options = {
  stages: [{ duration: '1m', target: 20 }, { duration: '2m', target: 100 }, {
duration: '1m', target: 0 }],
  thresholds: { http_req_failed: ['rate<0.01'], http_req_duration: ['p(95)
<1000'] },
};
export default function () { http.get('http://lb-01/search?q=plan'); }
```

탐지

번호	방식	내용
1000601	Suricata	한 출발지에서 `/search` 요청이 10초에 300회 초과 (규칙은 네트워크 탐지 담당에게 요청)
100160	Suricata 알림 능가	서명이 `INC-06`으로 시작하는 알림을 레벨 12로
100161	lb-01 HAProxy 로그	백엔드 서버가 `is DOWN` 상태가 됨

보강 (lb-01 `haproxy.cfg`의 frontend에 추가)

```
stick-table type ip size 100k expire 30s store http_req_rate(10s)
http-request track-sc0 src
http-request deny deny_status 429 if { sc_http_req_rate(0) gt 50 }
```

기대 결과: SECURE_MODE=0에서는 k6 임계치가 실패하고 1000601 알림이 뜬다. 보강 후에는 정상 사용자 요청의 오류율이 1% 이하가 되고, web-01을 꺼도 100161 알림만 뜨며 서비스는 유지된다. 측정 값은 메인 탭의 성공 지표 표에 옮긴다.

사고 대응 플레이북

알림이 뜨면 이 순서대로 대응하고, 티켓의 타임라인에 단계별 시각을 남긴다. 가장 중요한 원칙은 **증거를 먼저 보존하고 나서 지우거나 되돌린다**는 것이다. SKT 사고에서는 감염 서버 설정 변경이 자료 보전 위반으로, LG유플러스는 제보 뒤 서버 폐기·OS 재설치가 수사 의뢰로 이어졌다.

단계는 NIST SP 800-61 Rev.2의 사고 대응 주기를 따랐다. 2025년 4월에 나온 Rev.3는 같은 내용을 NIST CSF 2.0 체계로 다시 묶었다.

단계별 대응

단계	할 일	담당	산출물	시간 목표
1. 준비	연락망, 플레이북, baseline 스냅샷, 로그 보존 설정, 훈련	티켓·문서	이 문서, 스냅샷	2일차까지
2. 탐지·분석	알림 확인, 오탐 판별, 심각도 분류, 티켓 생성	해당 탐지 축 담당	티켓(열림)	알림 후 15분
3. 격리	VirtualBox 케이블 연결 해제 또는 ufw 차단, 관련 계정 잠금	해당 담당	티켓 조치 기록	분류 후 15분
4. 증거 보존	실행 중인 파일(`/proc/<PID>/exe`), 프로세스·소켓 목록, 로그, pcap, VM 스냅샷	해당 담당	`tickets/INC-0X/evidence/`	격리 직후, 제거 전
5. 제거·복구	원인 제거, 비밀값 교체, baseline 복원, 같은 시나리오로 재검증	해당 담당, 가용성·LB	재검증 결과	당일
6. 사후 활동	원인 분석, 규칙 보강, 교훈 정리, 티켓 종결	티켓·문서	종결된 티켓, 회고 기록	다음 날

심각도 기준

심각도	기준	예시
치명	숨은 백도어, 비밀값·개인정보 유출 가능성	INC-01, INC-02
높음	권한 우회로 대량 조회, 변조된 실행 파일	INC-03, INC-04
중간	미인가 장비 접속, 서비스 지연	INC-05, INC-06
낮음	단발성 스캐닝, 오탐으로 확인됨	-

국내 신고 기한 (훈련용)

이 프로젝트에서는 실제로 신고하지 않는다. 테이블탑 훈련에서 아래 기한 안에 누가 무엇을 하는지만 연습한다.

법령	대상	기한	신고처
정보통신망법 제48조의3	정보통신서비스 제공자의 침해사고	알게 된 때부터 24시간 이내 (2026년 10월 1일 시행 개정, 이전에는 "즉시")	과학기술정보통신부 또는 KISA
개인정보 보호법 제34조, 시행령 제40조	1천명 이상 유출, 민감·고유식별정보 유출, 외부 불법 접근에 의한 유출	알게 된 때부터 72시간 이내	개인정보보호위원회 또는 KISA

조문과 시행일은 검색으로 확인한 내용이다. 최종 보고서에 쓰기 전에 국가법령정보센터 원문으로 다시 확인한다.

테이블탑 훈련 (9일차, 30분씩)

진행자(티켓·문서 담당)가 상황을 단계별로 알려 주고, 나머지는 플레이북대로 말로 대응한다. 진행자는 시각과 결정을 기록한다.

1. **숨은 백도어**: web-01에서 삭제된 실행 파일이 돌고 있다는 알림(INC-01). 프로세스를 바로 죽일지, 증거부터 보존할지, 신고 대상인지 결정한다.
2. **비밀값 노출**: 팀원이 실수로 실제 API 키를 공개 저장소에 올렸다(가상 상황). 키 폐기, 히스토리 정리, 영향 범위 확인 순서를 정한다.
3. **서비스 장애**: INC-06 중 서비스가 40분간 멈춘다. 복구 우선순위와 사용자 공지 문구를 정한다.

각 훈련이 끝나면 플레이북에서 막혔던 점을 고치고, 고친 내용을 훈련 기록에 남긴다.

연락망

역할	이름	연락 수단	대리인
호스트 탐지			
네트워크 탐지			
가용성·LB			
티켓·문서 (사고 총괄)			

네트워크 설계

프로젝트망은 외부와 단절된 `10.10.10.0/24` 한 대역에 VM 5대를 고정 IP로 두고, 인터넷은 패키지 설치 단계에서만 NAT로 연결한 뒤 끊는다. 이 표의 IP·호스트명·포트는 팀 전체의 기준값이므로, 바꾸려면 문서 담당에게 요청한다.

하이퍼바이저는 VirtualBox를 기준으로 적었다. VMware를 쓰면 "호스트 전용 네트워크"를 VMnet(Host-only)으로 바꾸면 된다.

네트워크 2개

이름	종류	대역	용도	연결 시기
lab-net	호스트 전용 (Host-only), DHCP 끔	10.10.10.0/24	모든 프로젝트 트래픽, 호스트 PC에서 대시보드 접속	항상
setup-nat	NAT	자동	apt 설치, Wazuh·Suricata 내려받기	1~2일차 설치 때만, 2일차에 케이블 연결 해제

설치가 끝난 뒤 NAT를 끊는 이유는 부하 테스트와 공격 재현이 밖으로 나가는 일을 막기 위해서다.

VM 목록

호스트명	IP	OS	vCPU / RAM / 디스크	역할	열린 포트
attacker	10.10.10.10	Ubuntu 24.04 Server	1 / 2GB / 20GB	시나리오 재현, 부하 생성	없음
lb-01	10.10.10.20	Ubuntu 24.04 Server	1 / 1GB / 15GB	HAProxy, rate limit, fail2ban, Wazuh 에이전트	80/tcp
web-01	10.10.10.31	Ubuntu 24.04 Server	1 / 2GB / 20GB	Nginx 웹, Wazuh 에이전트, INC-01~03 대상	80/tcp (lb-01에서만)
web-02	10.10.10.32	Ubuntu 24.04 Server	1 / 2GB / 20GB	Nginx 웹, Wazuh 에이전트, 이중화	80/tcp (lb-01에서만)
monitor	10.10.10.50	Ubuntu 24.04 Server	4 / 8GB / 50GB	Wazuh 올인원, Suricata	443, 1514, 1515, 55000/tcp
rogue (INC-05 때만)	10.10.10.99	attacker 복제본	1 / 1GB	미인가 장비 흉내	없음

합계는 약 8 vCPU, 15GB RAM이다. monitor의 4 vCPU·8GB는 Wazuh 공식 권장 사양(에이전트 25개 이하)이다. 호스트 PC의 RAM이 16GB 이하라면 web-02와 lb-01을 Docker 컨테이너로 대신하거나, 팀원 PC 2대에 나눠 띄운다.

포트 사용표

포트	어디서 → 어디로	용도
80/tcp	attacker → lb-01	웹 요청(시나리오, 부하)
80/tcp	lb-01 → web-01, web-02	로드밸런싱
1514/tcp	web-01, web-02, lb-01 → monitor	Wazuh 에이전트 로그 전송
1515/tcp	에이전트 → monitor	Wazuh 에이전트 등록
443/tcp	호스트 PC → monitor	Wazuh 대시보드
55000/tcp	monitor 내부	Wazuh API
4444/tcp, 9999/udp	web-01 내부	INC-01 재현 전용(끝나면 종료)

방화벽 규칙 (ufw)

웹 서버는 lb-01에서 오는 80번만 받는다. 공격자가 웹 서버에 직접 붙지 못하게 하는 것이 망 분리의 최소 단위다(SK커뮤니케이션즈 사례의 교훈).

```
# web-01, web-02
sudo ufw default deny incoming
sudo ufw allow from 10.10.10.20 to any port 80 proto tcp
sudo ufw allow from 10.10.10.0/24 to any port 22 proto tcp
sudo ufw enable
```

```
# monitor
sudo ufw default deny incoming
sudo ufw allow from 10.10.10.0/24 to any port 1514,1515 proto tcp
sudo ufw allow from 10.10.10.1 to any port 443 proto tcp
sudo ufw allow from 10.10.10.0/24 to any port 22 proto tcp
sudo ufw enable
```

`10.10.10.1`은 VirtualBox 호스트 전용 어댑터의 호스트 PC 주소다. 어댑터 설정에서 이 값으로 맞춘다.

트래픽 미러링 (Suricata가 보는 방법)

monitor의 lab-net NIC를 무차별 모드로 두면, 같은 네트워크의 다른 VM 트래픽을 Suricata가 볼 수 있다.

1. VirtualBox → monitor 설정 → 네트워크 → 어댑터(lab-net) → 고급 → 무차별 모드: **모두 허용**
2. monitor 안에서 `sudo ip link set <NIC> promisc on`
3. attacker에서 lb-01로 `curl`을 보내고, monitor에서 `sudo tcpdump -i <NIC> host 10.10.10.20 and port 80`으로 보이는지 확인한다.

보이지 않으면 대안으로 Suricata를 lb-01에 설치한다. 모든 웹 트래픽이 lb-01을 지나기 때문에 INC-04·06은 그대로 탐지된다. INC-05의 ARP 감시는 arpwatch가 같은 대역 어느 VM에서나 동작하므로 영향이 없다.

기술 스택

탐지는 Wazuh(호스트)와 Suricata(네트워크) 두 축으로 하고, 가용성은 HAProxy·Nginx·fail2ban으로 대응한다. 모두 무료 오픈소스이고 Ubuntu 24.04에서 동작한다. 기준 버전은 Wazuh 4.14다.

도구 한눈에 보기

도구	분류	설치 위치	하는 일	쓰는 티켓
Wazuh 매니저·인덱서·대시보드	HIDS + SIEM	monitor	에이전트 로그를 받아 규칙으로 분석하고 알림을 띄움	INC-01~06 전체
Wazuh 에이전트	HIDS	web-01, web-02, lb-01	로그 수집, 명령 실행 결과 수집, 파일 변경 감시	INC-01~03
Suricata	NIDS	monitor (대안: lb-01)	패킷을 규칙과 비교해 공격 트래픽 탐지	INC-04, 06
arpwatch	네트워크 감시	monitor	새 MAC·IP 쌍이 나타나면 기록	INC-05
gitleaks	비밀값 스캐너	팀 PC, web-01	파일과 Git 히스토리에서 키·비밀번호 찾기	INC-02
Nginx	웹 서버	web-01, web-02	프로젝트용 웹 페이지 제공	INC-04, 06
HAProxy	소프트웨어 로드밸런서	lb-01	요청을 웹 서버 2대에 나눔, 죽은 서버 제외	INC-06
fail2ban	자동 차단	lb-01	로그에서 과도한 요청 IP를 찾아 방화벽으로 차단	INC-04, 06
k6	부하 테스트	attacker	정해진 양의 요청을 보내고 응답 시간·오류율 측정	INC-06
nmap	포트 스캐너	attacker	열린 포트 확인, 스캐닝 재현	INC-04

동작 원리

Wazuh: 각 서버의 에이전트가 로그와 명령 결과를 1514/tcp로 monitor에 보낸다. 매니저는 디코더로 로그를 해석하고, 규칙 레벨(0~15)에 따라 알림을 만든다. 알림은 인덱서에 저장되고 대시보드(443/tcp)에서 검색한다.

Suricata: NIC로 들어오는 패킷을 af-packet으로 받아 규칙과 비교한다. 탐지 결과는 `/var/log/suricata/eve.json`에 JSON으로 남고, Wazuh가 이 파일을 읽어 한 대시보드에서 함께 본다.

HAProxy: lb-01이 80번으로 요청을 받아 라운드로빈으로 web-01·web-02에 넘긴다. 주기적으로 각 서버를 점검해 응답이 없는 서버는 빼고 보낸다.

핵심 용어

용어	뜻
HIDS	서버 안에서 프로세스·파일·로그를 보는 탐지
NIDS	네트워크를 지나는 패킷을 보는 탐지
SIEM	여러 곳의 로그를 모아 상관 분석하고 알림을 만드는 시스템
FIM	파일 무결성 감시. 지정한 파일의 해시가 바뀌면 알림
규칙 ID / SID	Wazuh 규칙 번호 / Suricata 규칙 번호. 같은 번호를 두 명이 쓰면 충돌한다
BPF	커널 안에서 패킷을 걸러 보는 필터. BPFDoor는 이것으로 포트 없이 신호를 기다렸다
무차별 모드	자기 앞이 아닌 패킷도 받는 NIC 상태. 미러링에 필요
기준선	보강 전 정상 상태의 측정값. 전후 비교의 기준
ATT&CK	MITRE가 정리한 공격 기법 분류표. T로 시작하는 번호로 부른다

단계별 구축 가이드

VM 생성부터 탐지 확인까지 9단계로 나눠, 각 단계 끝의 "확인"이 통과해야 다음으로 넘어간다. IP와 호스트명은 네트워크 설계 탭을 따른다.

단계	내용	대상 VM	담당	예상 시간
0	호스트 PC 준비	호스트 PC	전원	30분
1	VM 생성과 공통 설정	전체	전원	2시간
2	Wazuh 설치	monitor	호스트 탐지	1시간
3	Wazuh 에이전트 등록	web-01, web-02, lb-01	호스트 탐지	30분
4	취약 웹앱 배포	web-01, web-02	가용성·LB	뼈대 반나절, 기능은 티켓별로
5	HAProxy 설정	lb-01	가용성·LB	30분
6	Suricata 설치와 Wazuh 연동	monitor	네트워크 탐지	1시간
7	arpwatch 설치	monitor	네트워크 탐지	20분
8	인터넷 차단과 스냅샷	전체	전원	20분

0. 호스트 PC 준비

1. VirtualBox와 Ubuntu 24.04 Server ISO를 받는다.
2. VirtualBox → 파일 → 도구 → 네트워크 관리자 → 호스트 전용 네트워크를 만들고, IPv4 `10.10.10.1`, 마스크 `255.255.255.0`, **DHCP 서버 끄기**로 설정한다.

확인: 호스트 PC에서 `ipconfig`를 실행하면 `10.10.10.1` 어댑터가 보인다.

1. VM 생성과 공통 설정

1. VM마다 어댑터 1은 NAT, 어댑터 2는 호스트 전용(lab-net)으로 만든다. 사양은 네트워크 설계 탭의 VM 목록을 따른다.
2. Ubuntu를 설치하면서 호스트명을 표의 이름 그대로 넣고, OpenSSH 서버를 함께 설치한다.
3. lab-net NIC에 고정 IP를 준다. NIC 이름은 `ip addr`로 확인한다(보통 `enp0s8`).

```
# /etc/netplan/60-lab.yaml  (web-01 예시, IP만 VM별로 바꾼다)
network:
  version: 2
  ethernets:
    enp0s8:
      addresses: [10.10.10.31/24]
```

```
sudo chmod 600 /etc/netplan/60-lab.yaml && sudo netplan apply
```

4. 모든 VM의 시간대와 `/etc/hosts`를 맞춘다. 시간이 어긋나면 티켓 타임라인과 로그 대조가 맞지 않는다.

```
sudo timedatectl set-timezone Asia/Seoul
sudo apt update && sudo apt -y upgrade
sudo apt -y install net-tools curl jq
printf '10.10.10.10 attacker\n10.10.10.20 lb-01\n10.10.10.31 web-
01\n10.10.10.32 web-02\n10.10.10.50 monitor\n' | sudo tee -a /etc/hosts
```

확인: 각 VM에서 `ping -c 2 monitor`와 `ping -c 2 web-01`이 성공한다.

1-1. 역할별 패키지 미리 설치

8단계에서 인터넷을 끊으면 apt와 pip가 동작하지 않는다. 2주 동안 쓸 도구는 지금 모두 설치한다.

VM	설치할 패키지	설치 단계
attacker	nmap, netcat-openbsd, k6	아래 명령
lb-01	haproxy, fail2ban, Wazuh 에이전트	3·5단계 + 아래 명령
web-01, web-02	nginx, python3-venv, netcat-openbsd, tcpdump, sqlite3, Wazuh 에이전트, MockTel 패키지	3·4단계 + 아래 명령
monitor	Wazuh 올인원, suricata, arpwatch	2·6·7단계
팀원 PC	gitleaks, pip-audit	프로젝트망 밖에서 저장소를 검사

```
# attacker: nmap, nc, k6 (k6는 Grafana 공식 저장소)
sudo apt -y install nmap netcat-openbsd gnupg
sudo gpg --no-default-keyring --keyring /usr/share/keyrings/k6-archive-keyring.gpg --keyserver hkp://keyserver.ubuntu.com:80 --recv-keys C5AD17C747E3415A3642D57D77C6C491D6AC1D69
echo "deb [signed-by=/usr/share/keyrings/k6-archive-keyring.gpg] https://dl.k6.io/deb stable main" | sudo tee /etc/apt/sources.list.d/k6.list
sudo apt update && sudo apt -y install k6
```

```
# lb-01
sudo apt -y install haproxy fail2ban
```

```
# web-01, web-02 (INC-01 재현에 nc·tcpdump, INC-02 점검에 sqlite3 사용)
sudo apt -y install nginx python3-venv netcat-openbsd tcpdump sqlite3
```

gitleaks와 pip-audit은 프로젝트망 VM이 아닌 팀원 PC에 설치한다. pip-audit은 온라인 취약점 DB를 조회해야 하고, gitleaks는 Git 저장소를 검사하기 때문이다.

확인: attacker에서 `k6 version`, `nmap --version`이 출력된다.

2. Wazuh 설치 (monitor)

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh && sudo bash ./wazuh-install.sh -a
```

1. 설치가 끝나면 화면에 admin 비밀번호가 나온다. 팀 비밀번호 관리자에만 저장하고, Git이나 채팅방에 올리지 않는다.
2. 호스트 PC 브라우저에서 `https://10.10.10.50`에 접속한다. 인증서 경고는 자체 서명 인증서라서 정상이다.

확인: 대시보드에 admin으로 로그인된다.

3. Wazuh 에이전트 등록 (web-01, web-02, lb-01)

1. 대시보드 → Agents → Deploy new agent에서 DEB amd64를 고르고, 서버 주소 `10.10.10.50`, 에이전트 이름에 호스트명을 넣는다.
2. 화면에 나온 설치 명령을 해당 VM에서 실행한 뒤 서비스를 켠다.

```
sudo systemctl daemon-reload && sudo systemctl enable --now wazuh-agent
```

확인: 대시보드 Agents 목록에 3대가 Active로 보인다.

4. 취약 웹앱 배포 (web-01, web-02)

가상의 통신사 고객센터 앱(가칭 MockTel)을 만들어, 사례별 취약점을 일부러 넣고 시나리오의 공격 대상으로 쓴다. 실제 통신사 이름과 로고는 쓰지 않는다.

기능	일부러 넣는 취약점	근거 사례	티켓	수정 버전
로그인	비밀번호를 평문으로 DB에 저장	SKT 2025	INC-02	argon2 해싱
설정 파일	DB 비밀번호와 API 키를 코드에 하드코딩	SKT 2025	INC-02	환경 변수로 분리
고객 조회 API	다른 고객 ID로도 조회됨(IDOR), 횟수 제한 없음	KT 2012·2014	INC-04	소유자 검사, rate limit
관리자 페이지	기본 계정 admin/admin	LG유플러스 2023	INC-04	기본 계정 제거
검색	제한 없는 무거운 조회	LG유플러스 2023	INC-06	페이지네이션, 캐시
의존성	취약점이 알려진 구버전 패키지 1개 고정	SK커뮤니케이션즈 2011	INC-03	최신 버전, pip-audit

앱은 Python Flask + SQLite로 만들고, 환경 변수 `SECURE_MODE=0/1`로 취약 버전과 수정 버전을 전환한다. 같은 시나리오를 두 번 돌려 전후를 비교하기 위해서다.

```
sudo mkdir -p /opt/mocktel && cd /opt/mocktel
python3 -m venv venv
# 인터넷이 연결된 지금 수정·취약 버전 패키지를 모두 받아 둔다 (버전이 달라 따로 받는다)
venv/bin/pip download -r requirements.txt -d wheels
venv/bin/pip download -r requirements-vuln.txt -d wheels
# 이후 설치는 항상 오프라인으로
venv/bin/pip install --no-index --find-links wheels -r requirements.txt
# gunicorn은 127.0.0.1:8000, Nginx가 80 → 127.0.0.1:8000 으로 넘긴다
```

확인: lb-01에서 `curl http://web-01/health`와 `curl http://web-02/health`가 모두 `ok`를 돌려준다. 웹 서버 2대는 응답에 자기 호스트명을 넣어 구분되게 한다.

5. HAProxy 설정 (lb-01)

```
sudo apt -y install haproxy
```

```
# /etc/haproxy/haproxy.cfg 끝에 추가
frontend web
    bind *:80
    default_backend mocktel

backend mocktel
    balance roundrobin
    option httpchk GET /health
    server web-01 10.10.10.31:80 check
    server web-02 10.10.10.32:80 check
```

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg && sudo systemctl restart haproxy
```

확인: attacker에서 `curl http://lb-01/health`를 여러 번 실행하면 web-01과 web-02가 번갈아 응답한다.

6. Suricata 설치와 Wazuh 연동 (monitor)

```
sudo apt -y install software-properties-common
sudo add-apt-repository -y ppa:oisf/suricata-stable
sudo apt update && sudo apt -y install suricata
sudo suricata-update
```

1. `/etc/suricata/suricata.yaml`에서 `HOME_NET`을 `"[10.10.10.0/24]"`로, `af-packet`의 `interface`를 lab-net NIC 이름으로 바꾼다.
2. 같은 파일의 `rule-files`에 팀 규칙 파일을 추가한다.

```
rule-files:
  - suricata.rules
  - /etc/suricata/rules/local.rules
```

3. Wazuh가 Suricata 결과를 읽도록 monitor의 `/var/ossec/etc/ossec.conf`에 추가하고 재시작한다.

```
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>
```

```
sudo systemctl restart suricata wazuh-manager
```

확인: 프로젝트망은 인터넷이 없으므로 공식 테스트 사이트 대신 팀 규칙으로 확인한다. `local.rules`에 아래 규칙을 넣고 attacker에서 `curl http://lb-01/lab-test`를 보내면 `/var/log/suricata/fast.log`와 Wazuh 대시보드에 알림이 떠야 한다.

```
alert http any any -> $HOME_NET 80 (msg:"COMMON: lab test rule"; http.uri;
content:"/lab-test"; sid:1000001; rev:1;)
```

7. arpwatch 설치 (monitor)

```
sudo apt -y install arpwatch
sudo systemctl enable --now arpwatch@enp0s8
```

확인: `journalctl -u arpwatch@enp0s8`에 프로젝트망 VM들의 MAC이 기록된다.

8. 인터넷 차단과 스냅샷

시작 전에 1-1단계의 패키지와 4단계의 MockTel `wheels/`가 모두 준비됐는지 확인한다. 이 단계는 2일차에 한다.

1. 모든 VM의 어댑터 1(NAT)에서 "케이블 연결" 체크를 해제한다.
2. 모든 VM을 끄고 `baseline`이라는 이름으로 스냅샷을 찍는다. 시나리오가 꼬이면 이 시점으로 되돌린다.

확인: attacker에서 `ping -c 2 8.8.8.8`이 실패하고, `curl http://lb-01/health`는 성공한다.

이후: 티켓 하나를 진행하는 순서

1. 작업 규약 탭에 따라 티켓 브랜치를 만든다.
2. `SECURE_MODE=0`, 기본 탐지만 켜고 재현한다. 탐지 결과를 캡처한다.
3. 탐지 규칙을 추가하고 다시 재현한다.
4. `SECURE_MODE=1`로 바꾸고 다시 재현해, 공격이 막히는지 확인한다.
5. 증거를 보존하고 티켓을 작성한 뒤 `baseline` 스냅샷으로 되돌린다.

작업 규약 (사람과 LLM 공통)

충돌을 막는 핵심은 세 가지다. 파일마다 주인이 한 명만 있고, 번호(규칙 ID, IP, 포트)는 미리 나눌 대역 안에서만 쓰며, 작업 전에 반드시 선점(claim)한다. 사람과 LLM 에이전트 모두 같은 규칙을 따르며, LLM용 전문은 저장소의 `AGENTS.md` 가 유일한 기준이다.

저장소 구조

```
security-lab/
├─ AGENTS.md                    모든 LLM 에이전트의 공통 규칙 (유일한 원본)
├─ CLAUDE.md, GEMINI.md      AGENTS.md를 가리키는 포인터
├─ .cursorrules                 상동
├─ .github/copilot-instructions.md   상동
├─ harness/                     규약 상세 (소유권, ID 대역, 작업 절차, 안전, 템플릿)
├─ docs/                        보고서, 플레이북
├─ .claims/                     작업 선점 파일 (INC-04.claim 등)
├─ app/
|  ├─ requirements.txt, requirements-vuln.txt
|  └─ mocktel/
|     ├─ core/                 공통 뼈대 (잠금)
|     └─ features/             기능별 파일, 티켓 담당자가 소유
├─ detection/
|  ├─ wazuh/rules/inc0X_rules.xml      티켓별 파일 분리
|  ├─ wazuh/agent/inc0X_localfile.xml
|  └─ suricata/inc0X.rules
├─ infra/                       hosts, netplan, ufw, haproxy, nginx, fail2ban
├─ scenarios/INC-0X/            reproduce.sh, cleanup.sh (INC-06은 k6 load.js)
├─ tests/postman/               MockTel 보안 테스트 컬렉션과 secure0·secure1 환경
└─ tickets/INC-0X/              ticket.md, worklog.md, evidence/
```

Wazuh는 `/var/ossec/etc/rules/` 안의 XML 파일을 모두 읽는다. 그래서 `local_rules.xml` 하나를 여러 명이 고치지 않고, 티켓마다 파일을 따로 둔다.

디렉터리 소유권

경로	주인	다른 사람·에이전트
`AGENTS.md` , `harness/` , `docs/`	티켓·문서	수정 금지, PR로 제안만
`app/mocktel/core/`	가용성·LB	수정 금지
`app/mocktel/features/<기능>.py`	해당 티켓 담당	수정 금지

경로	주인	다른 사람·에이전트
`detection/wazuh/`, `scenarios/INC-01~03/`	호스트 탐지	수정 금지
`detection/suricata/`, `scenarios/INC-04~05/`	네트워크 탐지	수정 금지
`infra/haproxy/`, `infra/fail2ban/`, `scenarios/INC-06/`	가용성·LB	수정 금지
`infra/netplan/`, `infra/ufw/`	티켓·문서	수정 금지
`tickets/INC-0X/`	해당 티켓 담당	`worklog.md`에 덧붙이기만 허용

주인이 아닌 파일을 고쳐야 하면 멈추고, 주인에게 요청한다. LLM도 같다.

번호 대역

티켓	Wazuh 규칙 ID	Suricata SID	재현용 포트
공통	100100–100109	1000001–1000099	–
INC-01	100110–100119	–	4444/tcp, 9999/udp
INC-02	100120–100129	–	–
INC-03	100130–100139	–	–
INC-04	100140–100149	1000400–1000499	–
INC-05	100150–100159	1000500–1000599	–
INC-06	100160–100169	1000600–1000699	–

IP와 호스트명은 네트워크 설계 탭의 값만 쓴다. 새 번호가 필요하면 티켓·문서 담당이 `harness/03-id-registry.md`에 등록한 뒤에 쓴다.

작업 절차

1. **시작**: `git pull` 후 `.claims/`를 확인한다. 같은 작업의 claim 파일이 있으면 멈춘다.
2. **선점**: `.claims/INC-0X.claim`을 만들어 주인·에이전트·시작 시각·수정할 경로를 적고 main에 바로 커밋·푸시한다. claim 파일만 담은 커밋은 main 직접 푸시를 허용하는 유일한 예외다. 파일명이 작업마다 다르므로 서로 충돌하지 않는다.

3. **브랜치**: `<역할>/<티켓>-<설명>` 형식. 예: `host/INC-01-packet-socket-rule`. claim 외에는 main에 직접 푸시하지 않는다.
4. **작업**: claim에 적은 경로만 고친다. 범위가 늘면 claim을 먼저 고친다.
5. **커밋**: `[INC-01] detect: 패킷 소켓 감시 규칙 추가` 형식. 종류는 claim(선점), `detect`, `scenario`, `app`, `infra`, `ticket`, `docs`. LLM이 작성한 커밋은 끝에 `Agent: <도구 이름>` 줄을 넣는다.
6. **기록**: `tickets/INC-0X/worklog.md` 끝에 한 항목을 덧붙인다. 기존 내용은 고치지 않는다.
7. **PR**: main 위로 rebase한 뒤 PR을 연다. 다른 사람 소유 파일에서 충돌이 나면 직접 해결하지 말고 주인에게 넘긴다. 머지는 사람이 리뷰한 뒤에만 한다.
8. **종료**: 머지되면 main에서 claim 파일을 지워 바로 푸시하고, 로드맵의 상태 칸을 갱신한다.

티켓이 없는 저장소 초기 설정은 `[SETUP]` 접두사를 쓴다(문서 담당만). 예: `[SETUP] docs: 하네스 추가`

LLM 에이전트 추가 규칙

- 세션을 시작할 때 항상 `AGENTS.md`와 `harness/` 전체를 읽는다.
- 사람이 지정한 티켓 하나만 맡는다. 스스로 다른 티켓을 시작하지 않는다.
- 잠긴 파일, 다른 사람 소유 파일, 번호 등록부는 고치지 않고 제안만 한다.
- 실제 악성코드, 프로젝트망 밖 대상, 실제 비밀값은 다루지 않는다. 취약점은 MockTel 안에만 둔다.
- 모르는 값(IP, 규칙 ID, 경로)은 추측하지 않고 등록부에서 찾거나 사람에게 물는다.
- 끝낼 때는 무엇을 바꿨고, 무엇을 확인했고, 무엇을 못 했는지 worklog에 남긴다.